

Connecting the Frontend to the Render API

How the React frontend was connected to the deployed FastAPI backend, published on Netlify, and prepared for Vercel.

Prepared: September 30, 2026 Repository: StratagemVX/spellSense-NLP_AAT

1. Final Architecture

FRONTEND

Netlify: `spellSense-nlp-aat.netlify.app`

React 19, TypeScript, Vite, Tailwind CSS

BACKEND

Render: `spellSense-nlp-aat-2.onrender.com`

FastAPI, Python, TextBlob and SymSpell

Request flow: Browser -> Netlify React application -> HTTPS fetch requests -> Render `/api` endpoints -> NLP correction response.

2. How the Connection Was Implemented

The frontend API service originally used the local development address when no environment variable was supplied:

```
http://127.0.0.1:8000/api
```

That address works only on the developer machine. It was changed to the deployed Render API as the production fallback:

```
const API_BASE_URL = import.meta.env.VITE_API_URL
|| 'https://spellSense-nlp-aat-2.onrender.com/api';
```

All frontend service calls build on this base URL:

- `POST /api/correct` for full text correction
- `POST /api/analyze-word` for single-word analysis
- `GET /api/examples` for example content
- `GET /api/health` for the navigation-bar service status

The Render backend already enabled CORS with `allow_origins=["*"]`, so browser requests from Netlify are accepted.

3. Netlify Deployment

1. Built the frontend from the `frontend` directory with `npm run build`.
2. Added `frontend/netlify.toml` with the build command and `dist` publish directory.
3. Added an SPA redirect so React Router routes resolve to `index.html` on refresh.
4. Created the Netlify project `spellsense-nlp-aat`.
5. Published the production build to Netlify.

```
[build]
  command = "npm run build"
  publish = "dist"

[[redirects]]
  from = "/*"
  to = "/index.html"
  status = 200
```

Live result: <https://spellsense-nlp-aat.netlify.app> returned HTTP 200 after deployment.

4. Vercel Preparation

A `frontend/vercel.json` file was added with the same single-page application rewrite:

```
{
  "rewrites": [
    { "source": "/(.*)", "destination": "/index.html" }
  ]
}
```

The Vercel CLI was installed, but no Vercel account credentials were available in the terminal session. Therefore the project is prepared for Vercel, but the final command requires account login:

```
cd frontend
vercel login
vercel --prod
```

5. Errors Encountered and Fixes

Error or issue	Cause	Fix
----------------	-------	-----

<code>npm ERR! enoent package.json</code>	The build was started from the repository root.	Changed into <code>frontend</code> before running <code>npm run build</code> .
Frontend would use localhost in production	The API fallback pointed to <code>127.0.0.1:8000</code> .	Changed the fallback to the Render HTTPS API URL.
React routes could fail after refresh	Netlify and Vercel need an SPA rewrite for client-side routes.	Added <code>netlify.toml</code> redirects and <code>vercel.json</code> rewrites.
Netlify site creation returned HTTP 404	The displayed team name was incorrectly passed as an account slug.	Removed the invalid <code>--account-slug</code> value and used the authenticated default team.
<code>netlify teams:list</code> was not recognized	The installed Netlify CLI version does not provide that command.	Used the supported site creation and deploy commands instead.
Vercel reported "No existing credentials"	The terminal was not authenticated to Vercel.	Added the Vercel configuration and documented the required <code>vercel login</code> step.
Potential browser CORS failure	Cross-origin frontend and backend deployments can be blocked by the browser.	Verified the backend has permissive CORS middleware; the deployed health request returned HTTP 200.

6. Validation Evidence

- Frontend TypeScript and Vite production build completed successfully.
- Render health check: `GET https://spellsense-nlp-aat-2.onrender.com/api/health` returned HTTP 200.
- Netlify live-site check: `https://spellsense-nlp-aat.netlify.app` returned HTTP 200.
- GitHub commit `407a0fe` contains the API connection and deployment configuration.

7. Final Deployment Checklist

Item	Status
------	--------

Render backend online	Complete
Frontend points to Render API	Complete
Frontend production build	Complete
Netlify production deployment	Complete
React Router refresh support	Complete
Vercel deployment	Ready after Vercel login

SpellSense NLP deployment documentation. The report describes the deployment state and validation performed on September 30, 2026.